INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT V

Functions & File Handling

V

Functions • Call by value / reference • Arrays as parameters • Scope & lifetime • File handling

▶ What you will learn in this unit

- Introduction to functions
- Declaration, definition & call
- Return types & arguments
- Call by value vs call by reference
- Modifying parameters with pointers
- Arrays as parameters
- Parameters & return values
- Scope of variables
- Lifetime & storage classes
- Basics of file handling
- fopen modes & file functions
- Exam questions

5.1 Introduction to Functions

◆ Definition

A **function** is a named, self-contained block of code that performs one specific task and can be **called** whenever needed. Every C program has at least one function: `main()`.

Benefit	Meaning
Modularity	Big problem → small functions (top-down design from Unit I)
Reusability	Write once, call many times
Easy debugging	Test and fix each function separately
Readability	Short main() that reads like a list of steps

Library functions	User-defined functions
Already written, declared in header files: `printf, scanf, sqrt, strlen, toupper`	Written by the programmer for a specific need: `add(), isPrime(), grade()`
Just `#include` the header and call	Must be declared, defined and called

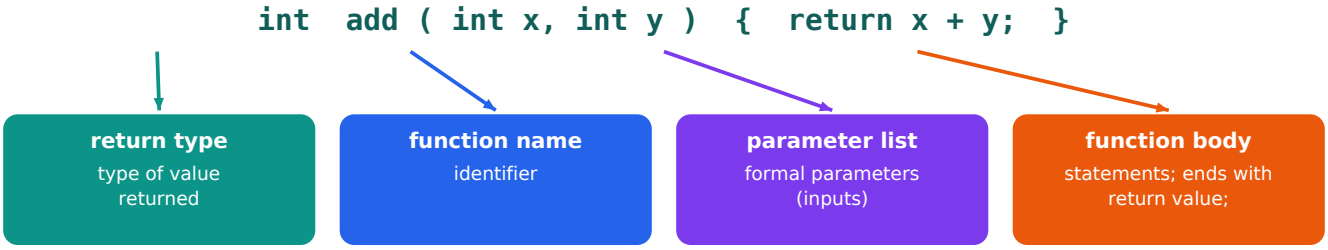


Fig 5.1 — Anatomy of a function definition

5.2 Function Declaration, Definition and Call

Step	What it is	Example
Declaration (prototype)	Tells the compiler the name, return type and parameter types **before** use; ends with ;	`int add(int, int);`
Definition	The actual code of the function	`int add(int x, int y) { return x + y; }`
Call	Transfers control to the function with **actual arguments**	`r = add(5, 3);`

Complete program with a user-defined function

```c
#include <stdio.h>
int add(int x, int y);            // 1. DECLARATION (prototype)

int main() {
    int r = add(5, 3);            // 3. CALL: 5 and 3 are ACTUAL arguments
    printf("Sum = %d", r);
    return 0;
}
int add(int x, int y) {           // 2. DEFINITION: x, y are FORMAL parameters
    return x + y;                 // sends the result back to the caller
}
```

OUTPUT

```
Sum = 8
```

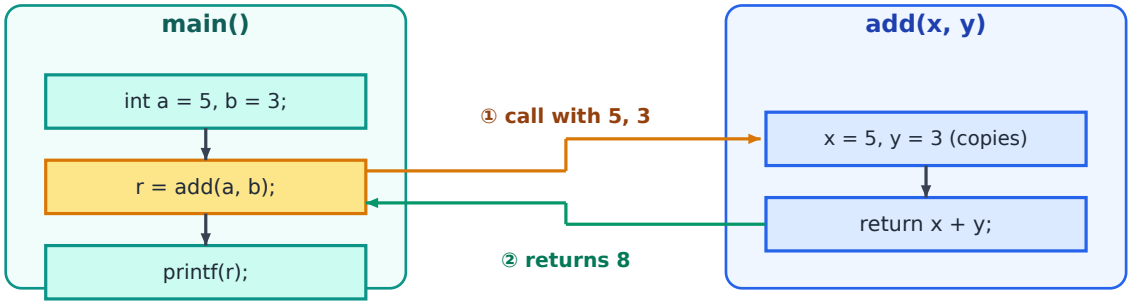


Fig 5.2 — Control flow of a function call: main → add → back to main

Return types and arguments — four kinds of functions

Type	Arguments	Return	Example
1	No	No	`void greet(void) { printf("Hello"); }`
2	**Yes**	No	`void square(int n) { printf("%d", n*n); }`
3	No	**Yes**	`int readNum(void) { int n; scanf("%d",&n); return n; }`
4	**Yes**	**Yes**	`int cube(int n) { return n*n*n; }` ← most useful

- **Return type**: `int, float, char, ...` or `void` (returns nothing). A function returns **one** value (use pointers for more).
- **Formal parameters** are in the definition; **actual arguments** are in the call. They match in number, order and type.
- **return** immediately ends the function. If the return type is not void, every path must return a value.
- If a function is defined **before** main(), the separate prototype is optional — but good practice.

A useful function: isPrime()

```c
int isPrime(int n) {
    int i;
    if (n < 2) return 0;                 // early return = "not prime"
    for (i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;                            // reached only if no divisor found
}
// in main:  printf("%d %d", isPrime(7), isPrime(10));
```

OUTPUT

```
1 0
```

5.3 Parameter Passing: Value vs Reference (Using Pointers)

Call by value: the function gets a **copy** of each argument — changes stay inside the function. **Call by reference:** the function gets the **address** (pointer) — it can change the caller's variables.

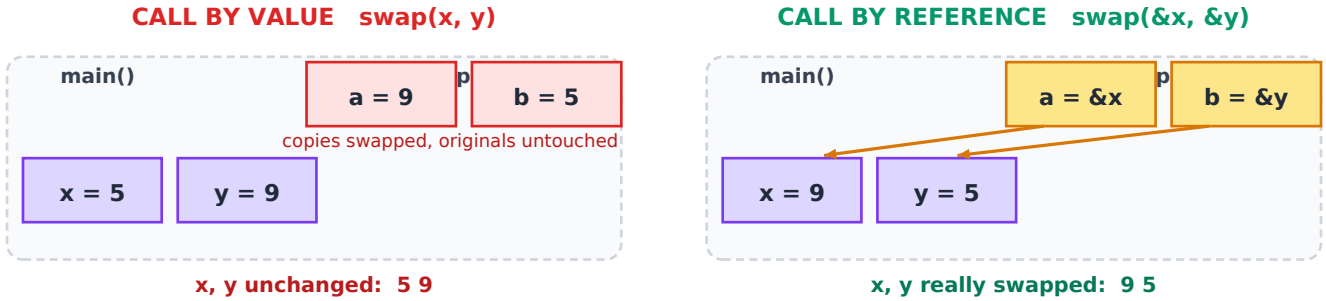


Fig 5.3 — Why swapping fails with values but works with pointers

Swap: by value and by reference

```c
void swapV(int a, int b)   { int t = a;  a = b;    b = t; }     // copies
void swapR(int *a, int *b) { int t = *a; *a = *b;  *b = t; }    // addresses

int main() {
    int x = 5, y = 9;
    swapV(x, y);    printf("%d %d\n", x, y);    // no effect
    swapR(&x, &y);  printf("%d %d\n", x, y);    // & passes the address
    return 0;
}
```

OUTPUT

```
5 9
9 5
```

	Call by value	Call by reference (pointers)
Passed	Copy of the value	Address of the variable
Formal parameter	Ordinary variable `int a`	Pointer `int *a`
Call	`f(x)`	`f(&x)`
Changes affect caller?	**No**	**Yes** (via `*a`)
Use for	Read-only inputs	Modify caller's data, return several results

Returning multiple values through pointers

```c
void minmax(int a[], int n, int *min, int *max) {   // returns TWO results
    int i;
    *min = *max = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] < *min) *min = a[i];
        if (a[i] > *max) *max = a[i];
    }
}
// main:  int a[] = {42,7,56,19,33}, lo, hi;
//        minmax(a, 5, &lo, &hi);   printf("Min=%d Max=%d", lo, hi);
```

OUTPUT

```
Min=7 Max=56
```

5.4 Arrays as Function Parameters

- Only the **address of the first element** is passed (no copy!), so the function works on the **original array** — it can modify it.
- The array **size is not passed** automatically — send it as a second argument `n`.
- `int a[]` and `int *a` mean exactly the same in a parameter list.
- For 2-D arrays the **column size must be given**: `void show(int m[][3], int rows)`.

Passing a 1-D array

```c
int sum(int a[], int n) {                // a[] == int *a
    int i, s = 0;
    for (i = 0; i < n; i++) s += a[i];
    return s;
}
void doubleAll(int a[], int n) {         // modifies the ORIGINAL array
    int i;
    for (i = 0; i < n; i++) a[i] *= 2;
}
// main:  int arr[] = {1,2,3,4,5};
//        printf("%d ", sum(arr, 5));       // call: array NAME, no []
//        doubleAll(arr, 5);
//        printf("%d", sum(arr, 5));
```

OUTPUT

```
15 30
```

Passing 2-D arrays and strings

```c
void show(int m[][3], int rows) {        // columns (3) must be specified
    int i, j;
    for (i = 0; i < rows; i++) {
        for (j = 0; j < 3; j++) printf("%d ", m[i][j]);
        printf("\n");
    }
}
int length(char s[]) {                   // strings too
    int n = 0;  while (s[n] != '\0') n++;  return n;
}
// show(matrix, 2);      length("Hello") -> 5
```

OUTPUT

```
1 2 3
4 5 6
```

5.5 Scope and Lifetime of Variables

Term	Meaning	Question it answers
Scope	The region of the program where a variable can be **accessed**	WHERE can I use it?
Lifetime	The period during **run time** for which the variable exists in memory	WHEN does it exist?

Kind	Declared	Scope	Lifetime	Default value
Local	Inside a function / block	Only that function / block	Until the block ends	Garbage
Global	Outside all functions	Whole file (after declaration)	Entire program run	0
Formal parameter	In function header	That function	During the call	Value of argument
static local	Local with `static`	That function only	**Entire program run**	0

Scope, shadowing and static

```c
int g = 10;                          // GLOBAL: visible everywhere below

void f() {
    int g = 20;                      // LOCAL g hides (shadows) global g
    printf("%d\n", g);
}
void counter() {
    int a = 0;  a++;                 // auto: recreated each call
    static int s = 0;  s++;          // static: created ONCE, remembers
    printf("a=%d s=%d\n", a, s);
}
// main: f(); printf("%d\n", g); counter(); counter(); counter();
```

OUTPUT

```
20
10
a=1 s=1
a=1 s=2
a=1 s=3
```

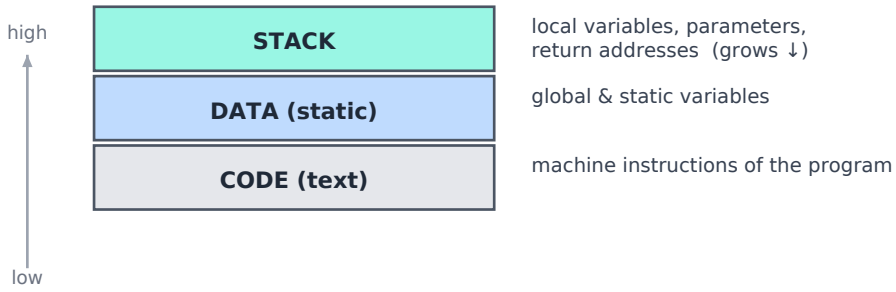


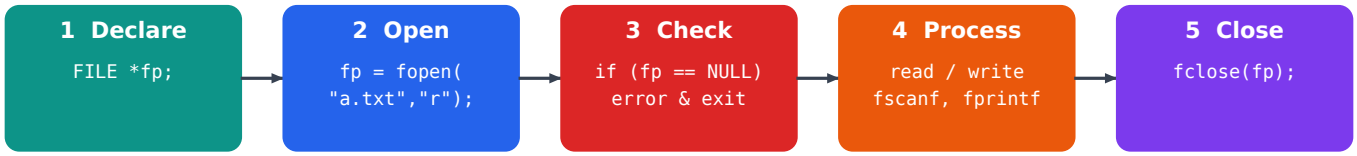
Fig 5.4 — Where variables live in memory (program layout)

Storage class	Keyword	Stored in	Default	Scope / Lifetime	Use
Automatic	`auto`	Stack	garbage	block / block	Normal local variables
Register	`register`	CPU register (hint)	garbage	block / block	Fast loop counters; cannot use &
Static	`static`	Data segment	0	block or file / **whole run**	Keep value between calls
External	`extern`	Data segment	0	global / whole run	Share a variable across files

5.6 Basics of File Handling

◆ Definition

A **file** stores data **permanently** on disk (variables vanish when the program ends). File handling lets a program **create, read, write and append** files using a **file pointer** of type `FILE *`.



Always follow these five steps — an unclosed file may lose its buffered data

Fig 5.5 — Steps in file processing



Fig 5.6 — Data flows through a buffer between program and disk

Mode	Meaning	If file does not exist	If file exists
`"r"`	Read only	**fopen returns NULL**	Opens at beginning
`"w"`	Write only	Creates new file	**Erases** old contents
`"a"`	Append (write at end)	Creates new file	Keeps data, adds at end
`"r+"` / `"w+"` / `"a+"`	Read and write	r+: NULL; w+, a+: create	w+ erases; a+ appends

Function	Purpose	Function	Purpose
`fopen(name, mode)`	Open file, returns FILE *	`fclose(fp)`	Close file
`fprintf(fp, fmt, ...)`	Formatted write	`fscanf(fp, fmt, ...)`	Formatted read
`fputc(c, fp)`	Write one char	`fgetc(fp)`	Read one char (EOF at end)
`fputs(s, fp)`	Write a string	`fgets(s, n, fp)`	Read a line
`feof(fp)`	True at end of file	`perror(msg)`	Prints reason for a failed open

Writing to a text file

```c
FILE *fp = fopen("data.txt", "w");          // "w": create / overwrite
if (fp == NULL) { printf("Cannot open file"); return 1; }
fprintf(fp, "Asha %d\n", 85);               // like printf, but into the file
fputs("Ravi 72\n", fp);
fclose(fp);                                 // data.txt now has 2 lines
```

OUTPUT

```
data.txt:
Asha 85
Ravi 72
```

Reading a text file

```c
char name[20];  int m;
fp = fopen("data.txt", "r");
while (fscanf(fp, "%s %d", name, &m) == 2)   // 2 items read?
    printf("%s scored %d\n", name, m);
fclose(fp);
```

OUTPUT

```
Asha scored 85
Ravi scored 72
```

Character I/O, counting lines, appending

```c
int ch, lines = 0;
fp = fopen("data.txt", "r");
while ((ch = fgetc(fp)) != EOF) {          // until end of file
    putchar(ch);
    if (ch == '\n') lines++;
}
fclose(fp);
fp = fopen("data.txt", "a");               // APPEND mode
fputs("Kiran 65\n", fp);  fclose(fp);
```

OUTPUT

```
Asha 85
Ravi 72
lines = 2
```

Write n numbers to a file, read them back, find the sum

```c
int n, i, x, sum = 0;
FILE *fp = fopen("nums.txt", "w");
printf("How many numbers? ");  scanf("%d", &n);
for (i = 0; i < n; i++) { scanf("%d", &x); fprintf(fp, "%d\n", x); }
fclose(fp);                                // numbers saved on disk
fp = fopen("nums.txt", "r");
while (fscanf(fp, "%d", &x) == 1) sum += x;   // read back
fclose(fp);
printf("Sum = %d", sum);
```

OUTPUT

```
How many numbers? 3
10 20 30
Sum = 60
```

★ Exam Focus

Remember: (1) always check `fp == NULL`; (2) mode `"w"` destroys old data; (3) `fclose()` flushes the buffer; (4) EOF is an **int** constant — store `fgetc()` result in `int`, not `char`; (5) `stdin, stdout, stderr` are automatically opened file pointers for keyboard, screen and error output.

More function examples

```c
void grade(int m) {                        // type 2: arguments, no return
    if (m >= 75)      printf("Distinction\n");
    else if (m >= 40) printf("Pass\n");
    else              printf("Fail\n");
}
long power(int b, int e) {                 // type 4: arguments + return
    long r = 1;  while (e-- > 0) r *= b;  return r;
}
// main: grade(82); grade(35); printf("%ld", power(2, 10));
```

OUTPUT

```
Distinction
Fail
1024
```

Copy a file and count characters / words

```c
FILE *in = fopen("data.txt", "r");
FILE *out = fopen("copy.txt", "w");
int ch, chars = 0, words = 0, inword = 0;
if (in == NULL || out == NULL) { perror("fopen"); return 1; }
while ((ch = fgetc(in)) != EOF) {
    fputc(ch, out);                                  // copy the file
    chars++;
    if (ch == ' ' || ch == '\n') inword = 0;
    else if (!inword) { inword = 1; words++; }   // new word
}
fclose(in);  fclose(out);
printf("chars=%d words=%d", chars, words);
```

OUTPUT

```
chars=16 words=4
```

Predict the output (practice)

#	Code	Output
1	`int f(int x){ return x*2; } printf("%d", f(f(3)));`	12
2	`void g(int a){ a = 99; } int x=5; g(x); printf("%d", x);`	5 (call by value)
3	`void h(int *a){ *a = 99; } int x=5; h(&x); printf("%d", x);`	99 (call by reference)
4	`void k(){ static int c=0; c++; printf("%d", c); } k(); k();`	12
5	`int a[3]={1,2,3}; void f(int x[]){ x[0]=9; } f(a); printf("%d", a[0]);`	9 (array passed by address)
6	`fopen("nofile.txt", "r")` when the file does not exist	returns NULL

Common mistake	Fix
Calling a function before declaring it	Add a prototype above main() or define it earlier
Forgetting `&` when the function expects a pointer	Call `swap(&x, &y)`, not `swap(x, y)`
Returning the address of a local variable	Local memory dies on return → dangling pointer; use a static variable
Not closing a file / not checking NULL	Always test `fp == NULL` and call `fclose(fp)`
Same name for a local and a global variable	The local hides the global (shadowing) — use distinct names

Stream	File pointer	Connected to	Example
Standard input	`stdin`	Keyboard	`fgets(s, 50, stdin);`
Standard output	`stdout`	Screen	`fprintf(stdout, "Hi");`
Standard error	`stderr`	Screen (error channel)	`fprintf(stderr, "Error!");`

● Real-life Use

Saving high scores and settings, student / bank records, log files, reading CSV data, configuration files, and exchanging data between programs.

★ UNIT SUMMARY — QUICK REVISION

- **Function** = reusable block: **declaration** (prototype), **definition**, **call**. Four kinds by arguments/return.
- **Call by value** copies; **call by reference** passes addresses (`f(&x)`, `int *p`) so the function can change the caller's variables.
- **Arrays** are passed as addresses (original is modified); pass the size separately; 2-D arrays need the column size.
- **Scope** = where visible (local / global); **lifetime** = how long it exists. `static` keeps its value between calls. Storage classes: auto, register, static, extern.
- **Files:** `FILE *fp` → `fopen` → check NULL → `fprintf/fscanf/fgetc/fputc/fgets/fputs` → `fclose`. Modes: r, w, a (+ b for binary).

Q Exam-Oriented Questions

Short Answer (2 marks)

1. What is a function? Mention two advantages.
2. Differentiate between function declaration and definition.
3. Differentiate between formal and actual parameters.
4. What is a function prototype? Why is it needed?
5. Differentiate between call by value and call by reference.
6. How are arrays passed to a function?
7. What is the difference between scope and lifetime?
8. What is a static variable? Give a use.
9. List the file opening modes in C.
10. What happens if `fopen()` fails to open a file?

Long Answer / Programs (5–10 marks)

1. Explain the types of user-defined functions based on arguments and return values, with examples.
2. Write a program to swap two numbers using call by value and call by reference. Explain the output.
3. Write a function to find the largest element of an array and call it from main().
4. Write functions to check even/odd and to compute a factorial using a loop; call them from main().
5. Explain local, global, static, auto, register and extern with examples.
6. Write a program to write n numbers to a file and then read them back and display the sum.
7. Explain file operations (open, read, write, append, close) with syntax and examples.
8. Write a program to count characters, words and lines in a file.

Σ Quick Reference: Syntax Cheat-Sheet

Concept	Syntax
Prototype	`return_type name(type1, type2);`
Definition	`return_type name(type1 p1, type2 p2) { ...; return value; }`
Call	`result = name(arg1, arg2);`
Call by reference	definition: `void f(int *p)` call: `f(&x);` use: `*p = 10;`
Array parameter	`int f(int a[], int n)` or `int f(int *a, int n)` call: `f(arr, 5)`
2-D array parameter	`void f(int m[][COLS], int rows)`
Static local	`static int count = 0;` (initialized once, keeps value)
Open / close file	`FILE *fp = fopen("f.txt", "r"); if (fp == NULL) ...; fclose(fp);`
Read / write text	`fscanf(fp, "%d", &n); fprintf(fp, "%d\n", n);`
Read until end	`while ((ch = fgetc(fp)) != EOF) { ... }`

Function checklist	Scope checklist
1. Prototype before main? 2. Argument count/types match? 3. Return type matches returned value?	Local: inside block, dies at } Global: whole file, lives till program ends Static: local scope, global lifetime